

Classification and Clustering for Comprehensive Obesity Data Analysis

Sara Miraglia (45669A)
sara.miraglia@studenti.unimi.it

Abstract

In this study, I leverage a comprehensive dataset to both predict obesity levels and uncover the underlying patterns that drive these outcomes. Using a combination of supervised and unsupervised machine learning approaches, I first build predictive models—including Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, and XGBoost—that classify individuals as obese or non-obese based on their demographic, behavioral, and dietary attributes. Then I employ Principal Component Analysis (PCA) and clustering (K-Means and Hierarchical clustering) to identify latent structures and groupings within the data.

Contents

1	Introduction	3
2	Dataset	3
2.1	Dataset Source	3
2.2	Dataset Description	3
2.3	Dataset Variables	3
3	Data Preparation	4
3.1	Data Cleaning	4
3.2	Renaming Columns	5
3.3	Feature Engineering	5
3.4	Handling Missing Values in Transformed Features	5
3.5	Correlation Analysis and Feature Removal	6
3.6	Scaling and Normalization	6
4	Descriptive Statistics	7
4.1	Distribution of Target Variable	7
4.2	Correlation Heatmap	7
5	Supervised Learning Analysis	9
5.1	Introduction	9
5.2	Methodologies overview and model trainig	9
5.3	Logistic Regression	10
5.4	K-Nearest Neighbors (KNN)	12
5.5	Decision Tree	13
5.6	Random Forest	15
5.7	XGBoost	17
5.8	Results	18
6	Unsupervised Learning Analysis	19
6.1	Introduction	19
6.2	Methodological Overview	19
6.3	Principal Component Analysis (PCA)	19
6.3.1	PCA Summary	19
6.3.2	Cumulative Variance Scree Plot	20
6.3.3	Visualization of Principal Component Loadings	21
6.3.4	Conclusion of PCA Analysis	21
6.4	PCA with k-Means Clustering	22
6.4.1	Elbow Method for Determining Optimal Clusters	22
6.4.2	K-Means Clustering Results	22
6.5	PCA with hierarchical Clustering	25
6.5.1	Dendrogram and Cluster Selection	25
6.5.2	Hierarchical Clustering Results	26
6.6	Results	27
7	R Code	28

1 Introduction

In this work, I apply a comprehensive analytical framework to identify the most critical predictors of obesity, build robust classification models, and discover hidden structures in the data. By integrating both supervised and unsupervised machine learning techniques, we aim to:

- Predict obesity levels using a variety of models, thereby determining which methods yield the highest accuracy and reliability.
- Explore underlying patterns in the data through dimensionality reduction and clustering, shedding light on subgroups and profiles that may not be immediately apparent.

First of all I begin with data processing, ensuring the dataset is clean, consistent, and ready for modeling. I then perform descriptive statistics and correlation analyses to gain initial insights into the relationships between key variables. Next, I train multiple classification models and compare their performance metrics, identifying the strongest predictors of obesity.

After this, there is an investigation on the data through PCA and clustering techniques to discover natural groupings. These groupings help characterize distinct risk profiles or behavioral patterns that correlate with obesity levels.

2 Dataset

2.1 Dataset Source

The dataset used in this project was sourced from the following publication:

Mendoza Palechor, F., & De la Hoz Manotas, A. (2019). *Dataset for estimation of obesity levels based on eating habits and physical condition in individuals from Colombia, Peru, and Mexico. Data in Brief*. Retrieved from <https://archive.ics.uci.edu/dataset/544/estimation+of+obesity+levels+based+on+eating+habits+and+physical+condition>

2.2 Dataset Description

The dataset contains information on obesity levels for individuals from Mexico, Peru, and Colombia, based on their eating habits and physical conditions. It includes 17 attributes across 2,111 records, with the target variable, *NObesity*, categorizing obesity into seven levels: Insufficient Weight, Normal Weight, Overweight Level I, Overweight Level II, Obesity Type I, Obesity Type II, and Obesity Type III. Approximately 23% of the data was collected from users via a web platform, while the remaining 77% was generated synthetically using Weka's SMOTE tool. This dataset provides a rich set of features for analyzing obesity trends, enabling both predictive and exploratory data analysis.

2.3 Dataset Variables

The dataset consists of the following variables:

Variable Name	Description
Gender	Gender of the individual.
Age	Age of the individual (in years).
Height	Height of the individual (in meters).
Weight	Weight of the individual (in kilograms).
Family_history with_overweight	Indicates if a family member has suffered or is suffering from overweight.
FAVC	Indicates if the individual frequently eats high-caloric food.
FCVC	Frequency of vegetable consumption during meals.
NCP	Number of main meals consumed daily.
CAEC	Indicates the frequency of food consumption between meals.
SMOKE	Indicates if the individual smokes.
CH2O	Amount of water consumed daily.
SCC	Indicates if the individual monitors their daily caloric intake.
FAF	Frequency of physical activity.
TUE	Time spent using technological devices (e.g., phone, TV, computer).
CALC	Frequency of alcohol consumption.
MTRANS	Primary mode of transportation used.
NObesidad	Obesity level, classified into seven categories: Insufficient Weight, Normal Weight, Overweight Levels I/II, and Obesity Types I/II/III.

Table 1: Description of Variables in the Dataset

3 Data Preparation

In this section, I describe the steps performed during the data preparation phase, which includes data cleaning, preprocessing, feature engineering, and scaling.

3.1 Data Cleaning

Data cleaning involves identifying and handling missing values and duplicates to ensure the integrity of the dataset. Missing values were identified and rows containing them were removed. Duplicate rows were also identified and removed to eliminate redundancy and maintain dataset consistency.

3.2 Renaming Columns

Column names were renamed for better readability and consistency. For instance, `Family_history_with_overweight` was renamed to `FamHist`, and `NObeyesdad` was renamed to `ObesityLevel`.

3.3 Feature Engineering

Several transformations were applied to the dataset to prepare it for numerical modeling. The following table provides detailed information on how the variables were modified during this step.

Variable Name	Transformation Applied
Gender	Converted to a binary variable: <code>Male</code> = 1, <code>Female</code> = 0.
FamHist	Converted to a binary variable indicating family history of overweight: <code>Yes</code> = 1, <code>No</code> = 0.
FAVC	Converted to a binary variable indicating frequent consumption of high-caloric food: <code>Yes</code> = 1, <code>No</code> = 0.
AlcoholCons	Encoded as an ordinal numeric variable based on frequency of alcohol consumption: <code>No</code> = 0, <code>Sometimes</code> = 1, <code>Frequently</code> = 2. Missing values were replaced with the median.
FoodFreq	Encoded as an ordinal numeric variable reflecting frequency of food consumption between meals: <code>Never</code> = 0, <code>Rarely</code> = 1, <code>Sometimes</code> = 2, <code>Frequently</code> = 3, <code>Always</code> = 4. Missing values were replaced with the median.
Transport	Converted to a binary variable indicating mode of transportation: <code>Active</code> (e.g., walking, biking) = 1, <code>Non-active</code> (e.g., public transportation, automobile) = 0. Missing values were replaced with 0.
ObesityLevel	Binarized to classify individuals into non-obese (0) and obese (1). Original categories grouped as follows: <code>Insufficient Weight</code> , <code>Normal Weight</code> , <code>Overweight Levels I/II</code> = 0; <code>Obesity Types I/II/III</code> = 1. Missing values were replaced with the median.

Table 2: Transformations Applied to Variables during Feature Engineering

These transformations ensured that the dataset was appropriately structured for numerical modeling, enabling effective analysis and predictive modeling.

3.4 Handling Missing Values in Transformed Features

Any missing values that resulted from transformations were addressed by replacing them with the median value of the respective feature, ensuring data completeness.

3.5 Correlation Analysis and Feature Removal

Special attention was given to the variable `Weight` since obesity levels are strongly influenced by weight. A correlation analysis showed a high correlation of 0.792 between `Weight` and `ObesityLevel`. Logistic regression confirmed that `Weight` is a strong predictor of `ObesityLevel`, with an accuracy of 89.31%. Despite its predictive power, `Weight` was removed from the dataset to avoid multicollinearity and to better understand the contributions of other features.

3.6 Scaling and Normalization

To ensure comparability across features, Min-Max scaling was applied to numeric variables, scaling their values to a range between 0 and 1.

4 Descriptive Statistics

In this section, the results of the descriptive statistical analysis performed on the dataset are presented.

4.1 Distribution of Target Variable

The distribution of the target variable was illustrated with a pie chart, offering a visual representation of the percentage of obese and non-obese individuals.

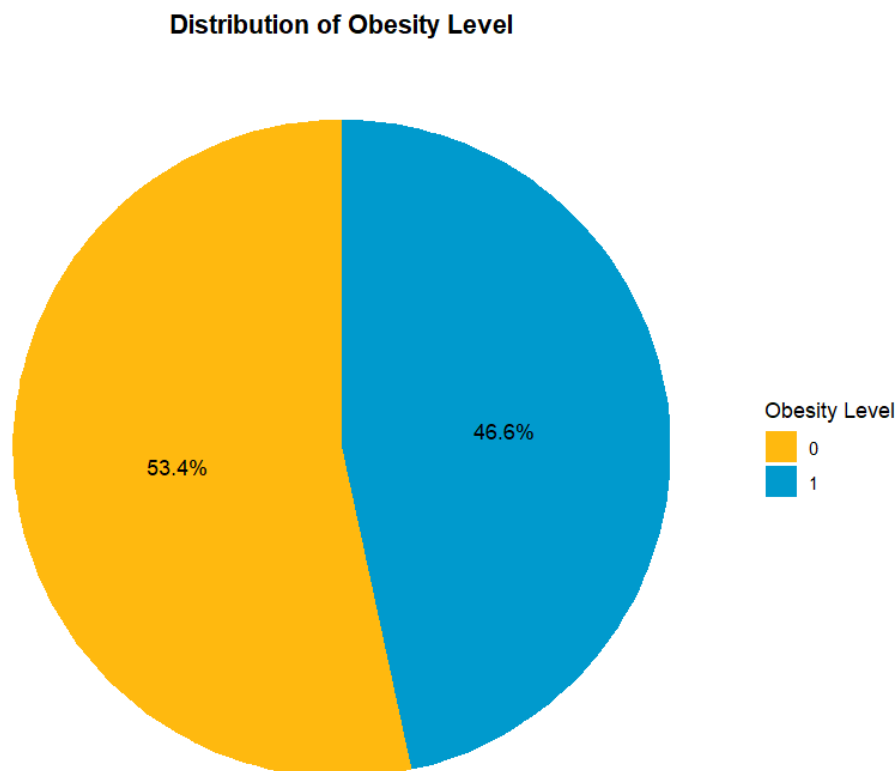


Figure 1: Distribution of Individuals Across Obesity Levels

The results show that the dataset is nearly balanced, with 53.43% of individuals classified as non-obese and 46.57% as obese. This near-equal distribution supports robust classification modeling while ensuring a realistic representation of the population.

4.2 Correlation Heatmap

A correlation heatmap was generated to visualize the relationships between the numeric variables in the dataset. Each cell in the heatmap represents the Pearson correlation coefficient between two variables, with values ranging from -1 to 1. Positive correlations are shown in warm colors (red and orange), while negative correlations are depicted in cool colors (blue and green).

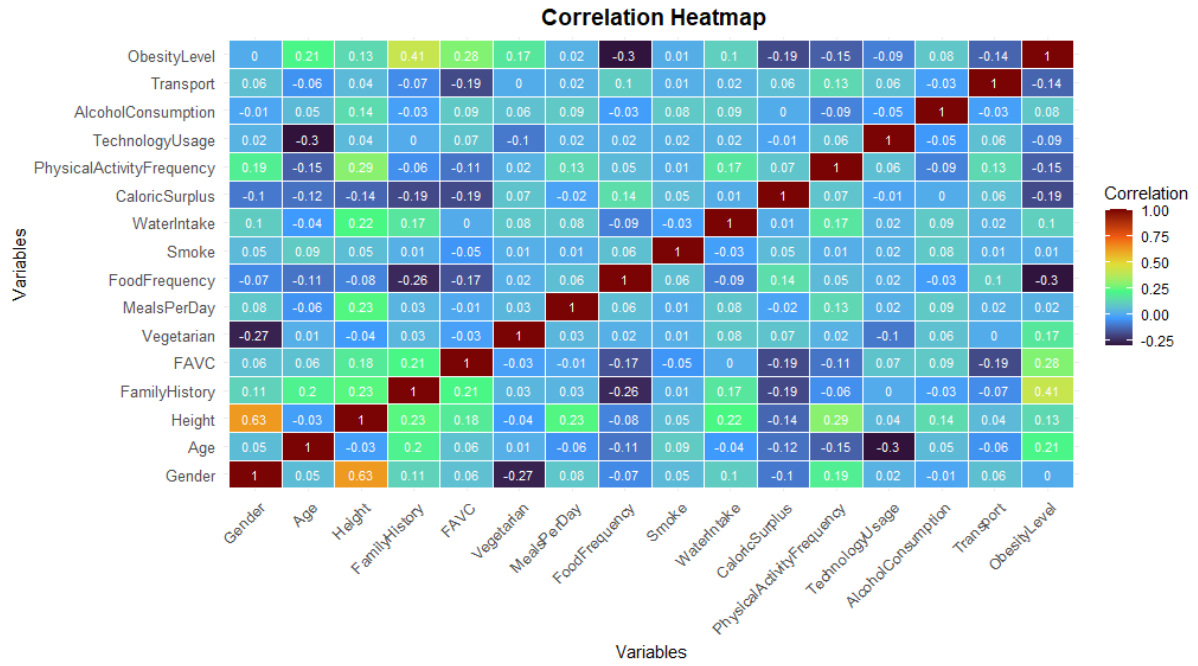


Figure 2: Correlation Heatmap of Numeric Variables. Warm colors indicate positive correlations, while cool colors indicate negative correlations. The intensity of the color reflects the strength of the correlation.

The variable most strongly associated with **ObesityLevel** is **FamilyHistory**, with a correlation coefficient of $r = 0.41$, indicating a moderately strong positive relationship. This suggests that individuals with a family history of obesity are more likely to be obese.

Another notable variable is **FAVC** (Frequent Consumption of High-Calorie Foods), which has a correlation of $r = 0.28$, indicating that frequent intake of energy-dense foods is positively linked to obesity.

Similarly, **Age** shows a positive correlation of $r = 0.21$, suggesting that older individuals might be more prone to obesity. Additionally, **CaloricSurplus** is negatively correlated with **ObesityLevel** at $r = -0.19$, implying that individuals who maintain a calorie balance are less likely to be obese.

5 Supervised Learning Analysis

5.1 Introduction

In the supervised learning phase of this study, the objective was to evaluate and compare the effectiveness of various predictive modeling techniques in classifying obesity levels.

5.2 Methodologies overview and model trainig

Initially, the dataset was partitioned into training and testing subsets, allocating 80% of the data to the training set and the remaining 20% to the testing set. This split was executed using a randomized approach with a set seed to ensure reproducibility and eliminate potential biases. The training set was utilized to develop and tune the models, while the testing set served to evaluate their performance on unseen data.

The training process involved the following steps for each model:

- **Logistic regression:** A logistic regression model was constructed using the training set. This model, suitable for binary outcomes, allowed us to estimate the probabilities of bipolar obesity.
- **K-Nearest Neighbors (KNN):** The optimal number of neighbors was determined through cross-validation, enabling the model to adapt to the underlying data structure without imposing strict assumptions about the data distribution.
- **Decision Tree:** The maximum depth of the tree was tuned using cross-validation to control the model's complexity, ensuring that it captured essential patterns without overfitting the training data.
- **Random Forest:** Parameter tuning focused on the number of variables considered at each split (`mtry`) and the number of trees (`ntree`), aiming to enhance predictive accuracy through ensemble learning.
- **XGBoost:** Hyperparameters such as learning rate (`eta`), maximum tree depth (`max_depth`), and subsample ratio were adjusted using cross-validation to prevent overfitting and optimize model performance.

The performance metrics used for each methods include Accuracy, Precision, Recall, and F1 Score were calculated to assess and compare the models' effectiveness in classifying obesity levels.

5.3 Logistic Regression

A logistic regression model was used to identify which variables have a significant impact on predicting **ObesityLevel**. This method, known for its ability to handle binary classification problems, was chosen to highlight the most important predictors and assess their relevance. The analysis provides a clear understanding of variable relationships and serves as a foundation for evaluating predictive models in addressing obesity-related issues.

```
Call:
glm(formula = ObesityLevel ~ ., family = binomial(), data = train_set_lr)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -7.0588     0.6069 -11.631 < 2e-16 ***
Gender        -0.2278     0.1732  -1.315 0.188566
Age           1.9848     0.5168   3.841 0.000123 ***
Height        1.1169     0.5583   2.000 0.045447 *
FamHist       3.6231     0.4266   8.492 < 2e-16 ***
FAVC          2.4312     0.3307   7.352 1.95e-13 ***
VegCons       1.5923     0.2583   6.165 7.06e-10 ***
Meals        -0.0378     0.2612  -0.145 0.884941
FoodFreq     -3.6386     0.5126  -7.098 1.27e-12 ***
Smoke         0.9052     0.4876   1.857 0.063378 .
WaterIntake   0.2926     0.2243   1.304 0.192173
CaloricSurplus -2.6049     0.7565  -3.443 0.000575 ***
PhysActFreq  -1.2100     0.2507  -4.826 1.39e-06 ***
TechUsage    -0.3323     0.2167  -1.533 0.125267
AlcoholCons  -0.0243     0.2580  -0.094 0.924968
Transport    -1.8841     0.6699  -2.812 0.004919 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 2307.3  on 1669  degrees of freedom
Residual deviance: 1579.4  on 1654  degrees of freedom
AIC: 1611.4

Number of Fisher Scoring iterations: 6
```

Figure 3: Logistic Regression Coefficients

The logistic regression analysis identified several significant predictors of **ObesityLevel**, highlighting their respective roles in influencing obesity outcomes:

- **Family History (FamHist):** This variable had a highly positive and statistically significant coefficient (p-value < 0.001), indicating that individuals with a family history of obesity are more likely to experience higher obesity levels. This finding emphasizes the critical role of genetic predisposition in obesity-related outcomes.
- **Frequent Consumption of High-Calorie Foods (FAVC):** A significant positive coefficient (p-value < 0.001) suggests that frequent consumption of high-calorie foods increases the likelihood of obesity. This result underscores the importance of dietary habits in managing weight.
- **Food Frequency (FoodFreq):** This variable showed a strong and negative relationship with obesity levels (p-value < 0.001), implying that increased meal frequency, when associated with smaller portions, may reduce obesity risk.

- **Physical Activity Frequency (PhysActFreq):** The significant negative coefficient (p-value < 0.001) indicates that higher physical activity frequency is associated with reduced obesity levels. This highlights the importance of regular exercise in maintaining a healthy weight.
- **Caloric Surplus (CaloricSurplus):** A negative and significant coefficient (p-value = 0.0006) reveals that a caloric surplus significantly contributes to an increased risk of obesity, emphasizing the importance of caloric balance.
- **Age and Height:** These variables were also significant predictors (p-values < 0.05), suggesting that age and height may influence obesity levels, potentially through metabolic or developmental factors.

Other variables, such as **Water Intake** and **Alcohol Consumption**, were not statistically significant, indicating that their variation may have a less direct or weaker effect on obesity levels within the context of this model.

MODEL	ACCURACY	PRECISION	RECALL	F1
Logistic Regression	0.7818	0.8837	0.6816	0.7696

Table 3: Performance Metrics for Logistic Regression Model

5.4 K-Nearest Neighbors (KNN)

The analysis involved a structured approach to data preparation, model training, and evaluation. K-Nearest Neighbors (KNN) is a non-parametric technique that can effectively identify intricate patterns within the data without requiring a predefined model structure. This flexibility makes it particularly useful for addressing nonlinear relationships between variables. The KNN algorithm was employed to predict obesity levels.

To ensure the robustness of the model, it was trained using 10-fold cross-validation, combined with hyperparameter tuning to determine the optimal value of k .

k-Nearest Neighbors

```
1670 samples
 15 predictor
 2 classes: 'NonObese', 'Obese'

Pre-processing: centered (15), scaled (15)
Resampling: Cross-Validated (10 fold)
Summary of sample sizes: 1502, 1503, 1504, 1503, 1503, 1503, ...
Resampling results across tuning parameters:
```

k	Accuracy	Kappa
5	0.9131874	0.8264432
7	0.9089993	0.8182250
9	0.9012256	0.8027211
11	0.9012291	0.8029248
13	0.8910530	0.7828466
15	0.8832614	0.7674723
17	0.8754841	0.7521899
19	0.8664948	0.7346463
21	0.8676817	0.7371189
23	0.8527079	0.7077784
25	0.8521019	0.7066499
27	0.8449305	0.6922344
29	0.8329473	0.6688746
31	0.8293509	0.6614420
33	0.8221688	0.6470787
35	0.8197700	0.6423657
37	0.8203652	0.6437028
39	0.8149688	0.6331952
41	0.8095796	0.6226791
43	0.8071879	0.6180476

Accuracy was used to select the optimal model using the largest value.
The final value used for the model was $k = 5$.

Figure 4: K-Nearest Neighbors (KNN) Model Performance

MODEL	ACCURACY	PRECISION	RECALL	F1
KNN	0.8897	0.9234	0.8655	0.8935

Table 4: Performance Metrics for K-Nearest Neighbors (KNN) Model

5.5 Decision Tree

The next step of the analysis involved implementing a Decision Tree model, trained using 10-fold cross-validation to ensure robust performance evaluation. During this process, the *maxdepth* parameter, which determines the maximum depth of the tree, was carefully tuned to optimize the model's performance.

```
CART
1670 samples
  15 predictor
    2 classes: 'NonObese', 'Obese'

No pre-processing
Resampling: Cross-Validated (10 fold)
Summary of sample sizes: 1502, 1503, 1503, 1503, 1503, 1504, ...
Resampling results across tuning parameters:

maxdepth  Accuracy  Kappa
3          0.7245203  0.4618657
5          0.7976358  0.5953707
7          0.8096118  0.6201815
10         0.8191785  0.6385617
15         0.8293510  0.6579611
```

Accuracy was used to select the optimal model using the largest value.
The final value used for the model was *maxdepth* = 15.

Figure 5: Decision Tree Structure

Decision Tree with Optimized Maxdepth

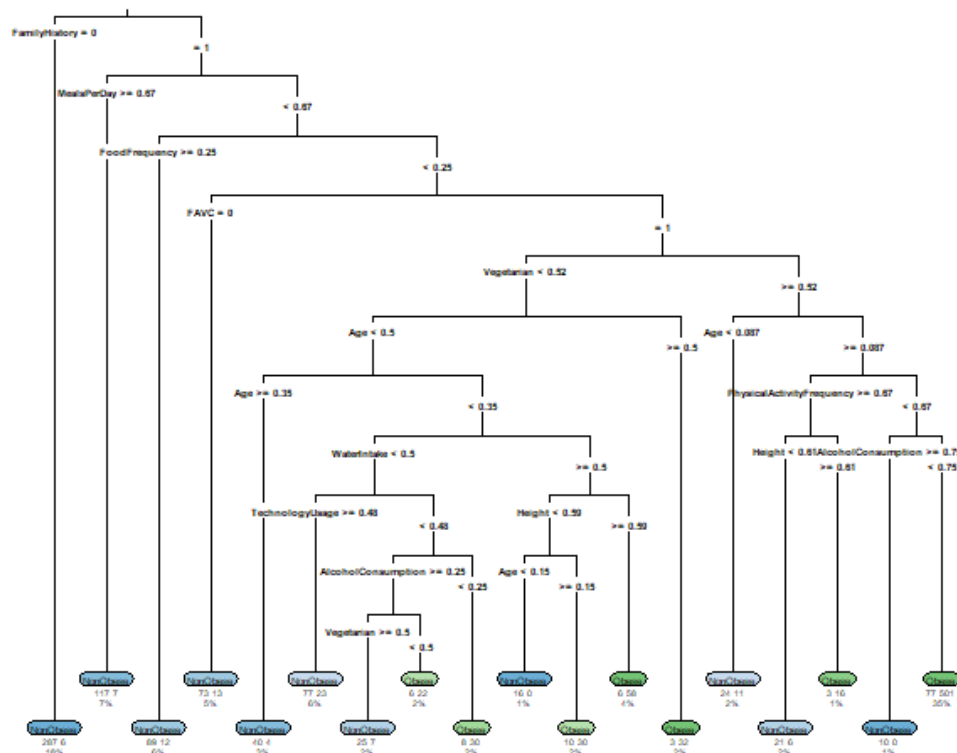


Figure 6: Decision Tree Performance Metrics

MODEL	ACCURACY	PRECISION	RECALL	F1
Decision Tree	0.8513	0.8779	0.8386	0.8578

Table 5: Performance Metrics for Decision Tree Model

5.6 Random Forest

Building upon the Decision Tree model, a Random Forest model was implemented to enhance predictive performance through ensemble learning. Random Forests aggregate the predictions of multiple decision trees to improve accuracy and control overfitting.

The Random Forest model was trained using 5-fold cross-validation to ensure robust performance evaluation. Hyperparameter tuning focused on selecting the optimal number of variables considered at each split (`mtry`) and the number of trees (`ntree`) to balance bias and variance effectively. A total of 500 trees were grown to ensure sufficient model complexity and stability.

To understand the factors most influential in predicting obesity levels, a variable importance analysis was conducted. This analysis ranks the features based on their contribution to the model's predictive power.

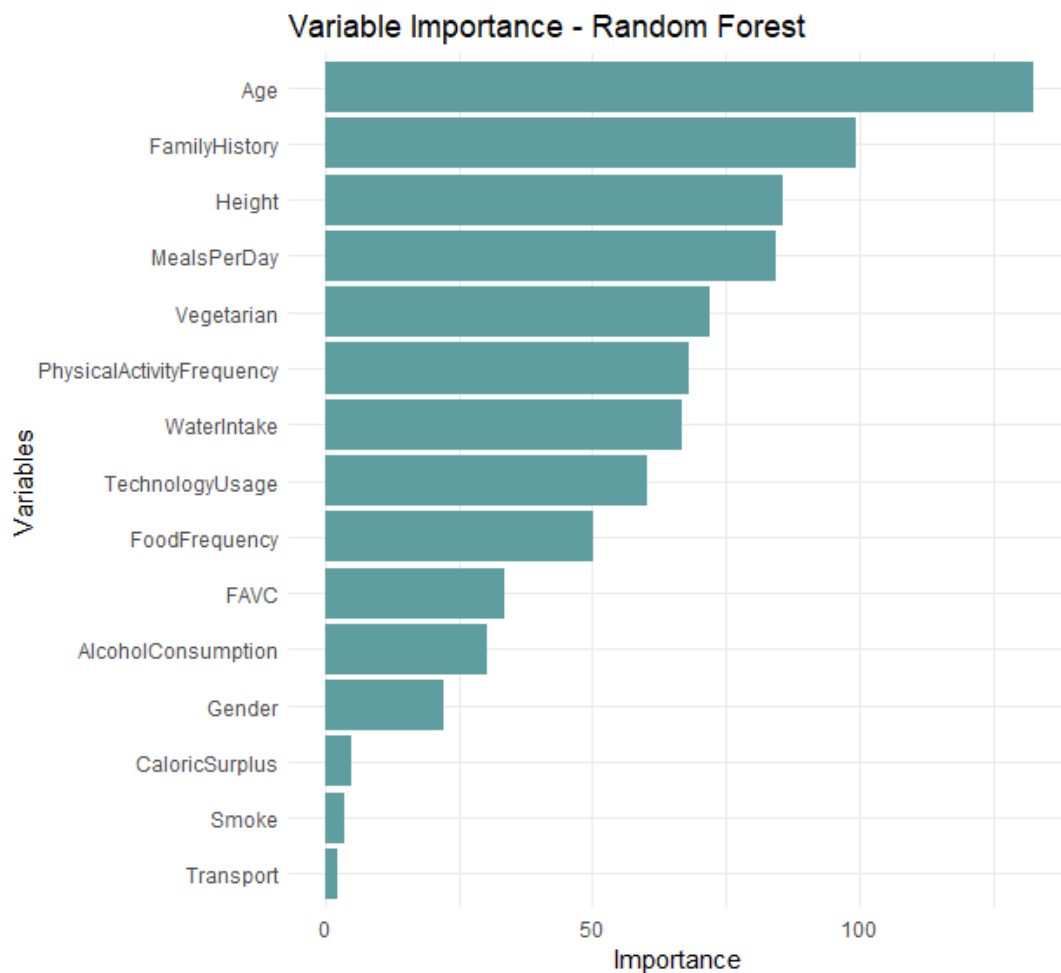


Figure 7

Age stands out as the most influential predictor, with a significantly higher importance score compared to other factors. **Family History** and **Height** are the next most impactful variables, followed by **Meals Per Day** and whether the individual is **Vegetarian**.

Other moderately important factors include **Physical Activity Frequency**, **Water Intake**, and **Technology Usage**, indicating their relevance in the model’s predictive capacity. Less impactful variables include **Alcohol Consumption**, **Gender**, and **Caloric Surplus**, while **Smoking** and **Transport** contribute minimally to the model’s predictions.

Overall, the results highlight that demographic factors (like **Age** and **Family History**) and lifestyle choices (such as **Meals Per Day** and **Activity Levels**) play critical roles in the model’s predictions.

MODEL	ACCURACY	PRECISION	RECALL	F1
Random Forest	0.9400	0.9541	0.9327	0.9433

Table 6: Performance Metrics for Random Forest Model

5.7 XGBoost

In the end, the Extreme Gradient Boosting (XGBoost) algorithm was employed to further enhance predictive accuracy. XGBoost is renowned for its efficiency and effectiveness in handling large datasets and complex relationships through gradient boosting.

The model was trained using 5-fold cross-validation to ensure robust performance evaluation. A comprehensive hyperparameter tuning grid was defined to explore various combinations of parameters, including the number of boosting rounds (**nrounds**), maximum tree depth (**max_depth**), learning rate (**eta**), minimum loss reduction (**gamma**), subsample ratio of columns (**colsample_bytree**), minimum sum of instance weight needed in a child (**min_child_weight**), and subsample ratio of the training instances (**subsample**). This extensive search aimed to identify the optimal set of parameters that balance bias and variance, thereby preventing overfitting and enhancing generalization.

After exhaustive tuning, the best-performing parameters were identified as follows:

Parameter	Value
nrounds	100
max_depth	9
eta	0.1
gamma	0
colsample_bytree	0.9
min_child_weight	1
subsample	0.9

Table 7: Best Tuning Parameters for XGBoost Model

The optimized XGBoost model was then evaluated on the testing set, yielding the following performance metrics:

MODEL	ACCURACY	PRECISION	RECALL	F1
XGBoost	0.9448	0.9587	0.9372	0.9478

Table 8: Performance Metrics for XGBoost Model

5.8 Results

Model	Accuracy	Precision	Recall	F1_Score
Logistic Regression	0.7818	0.8837	0.6816	0.7696
KNN	0.8897	0.9234	0.8655	0.8935
Decision Tree	0.8513	0.8779	0.8386	0.8578
Random Forest	0.9400	0.9541	0.9327	0.9433
XGBoost	0.9448	0.9587	0.9372	0.9478

Figure 8

The evaluation metrics indicate that the XGBoost model outperforms the other models across all key metrics:

- **Accuracy:** XGBoost achieved the highest accuracy of 0.9448, indicating its overall correctness in predictions.
- **Precision:** With a precision of 0.9587, XGBoost demonstrates strong reliability in positive predictions.
- **Recall:** A recall of 0.9372 shows that XGBoost effectively identifies positive cases.
- **F1 Score:** The F1 Score of 0.9478 balances precision and recall, indicating superior overall performance.

So, the XGBoost model is the best-performing model.

6 Unsupervised Learning Analysis

6.1 Introduction

Unsupervised learning techniques were employed in this study to understand the hidden patterns and structures within the dataset, providing deeper insights into the relationships among features and groups of individuals.

6.2 Methodological Overview

The focus of the analysis was on dimensionality reduction and clustering, leveraging a combination of Principal Component Analysis (PCA), K-Means clustering, and Hierarchical clustering to explore the dataset.

Principal Component Analysis (PCA) was utilized to reduce the dimensionality of the dataset.

Following PCA, two clustering techniques were applied to identify meaningful groupings:

- **K-Means Clustering:** This method was employed to partition the dataset into clusters by capturing central tendencies and grouping similar data points.
- **Hierarchical Clustering:** This technique was used to uncover nested relationships and hierarchical groupings within the data, offering insights into the layered structure and potential subgroups within the clusters identified by K-Means.

This integrated approach—combining PCA for dimensionality reduction with K-Means and Hierarchical clustering for structural exploration—facilitated a comprehensive understanding of the dataset’s intrinsic groupings and relationships.

6.3 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a robust dimensionality reduction technique utilized to simplify complex datasets by transforming them into a set of linearly uncorrelated variables known as principal components. The primary aim of PCA is to capture the maximum variance present in the data with the fewest number of components, thereby reducing dimensionality while preserving as much information as possible.

6.3.1 PCA Summary

The PCA was conducted on the scaled dataset excluding the target variable `ObesityLevel`. The summary of the PCA provides detailed information on the standard deviation, proportion of variance explained, and cumulative proportion of variance for each principal component.

Importance of components:								
	Comp.1	Comp.2	Comp.3	Comp.4	Comp.5	Comp.6	Comp.7	Comp.8
Standard deviation	1.4672958	1.3350538	1.15033999	1.11292072	1.06017718	1.00071874	0.97020457	0.95808920
Proportion of Variance	0.1435305	0.1188246	0.08821881	0.08257284	0.07493171	0.06676253	0.06275313	0.06119566
Cumulative Proportion	0.1435305	0.2623550	0.35057384	0.43314668	0.50807839	0.57484092	0.63759405	0.69878971
	Comp.9	Comp.10	Comp.11	Comp.12	Comp.13	Comp.14	Comp.15	
Standard deviation	0.90679830	0.88564423	0.87707732	0.81808215	0.78583161	0.76009855	0.52697380	
Proportion of Variance	0.05481888	0.05229105	0.05128431	0.04461723	0.04116876	0.03851665	0.01851343	
Cumulative Proportion	0.75360858	0.80589963	0.85718394	0.90180117	0.94296992	0.98148657	1.00000000	

Figure 9: Summary of Principal Component Analysis

Table 9 presents the standard deviation, proportion of variance, and cumulative proportion of variance for each principal component. The first ten principal components cumulatively explain approximately 80.59% of the total variance in the dataset. This indicates that a significant portion of the data's variability can be captured with a reduced number of components, thereby simplifying subsequent analyses without substantial loss of information.

6.3.2 Cumulative Variance Scree Plot

This plot illustrates the cumulative percentage of variance explained as each additional principal component is included.

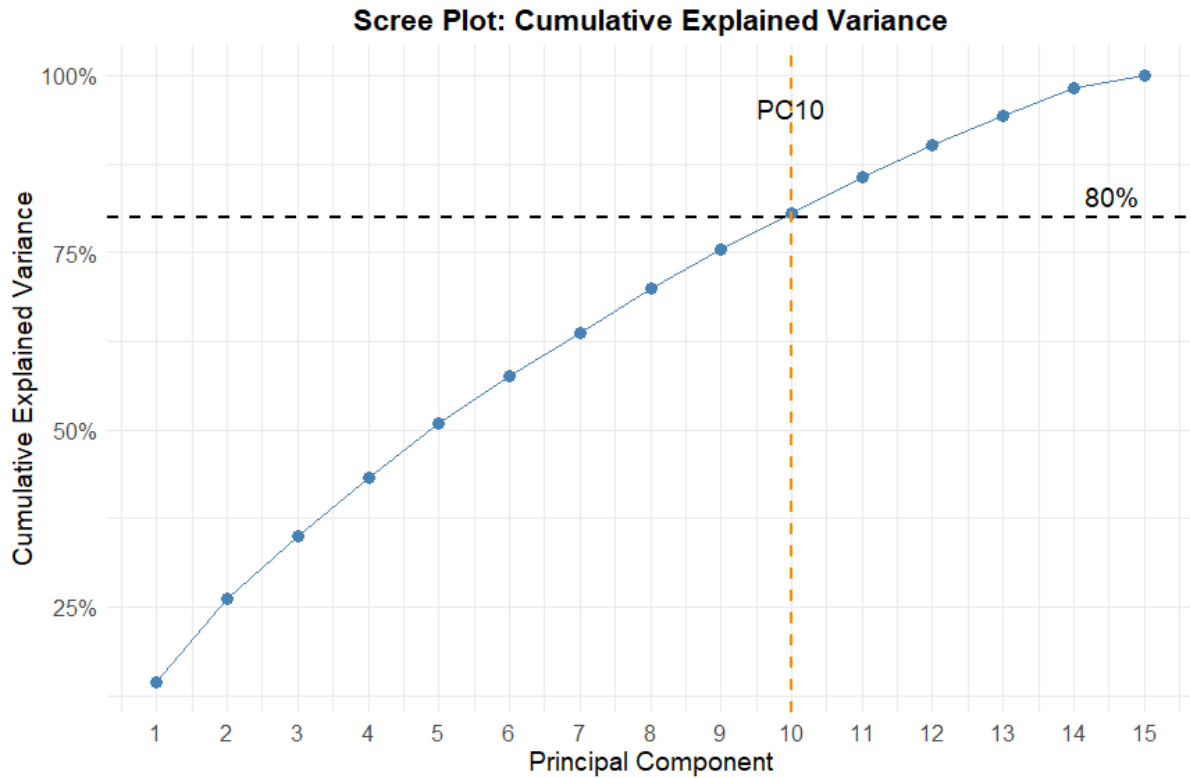


Figure 10: Cumulative Variance Scree Plot for Principal Component Analysis

The scree plot serves as a visual tool to assess the amount of variance captured by the principal components. By plotting the cumulative variance against the number of components, it becomes easier to identify the point at which additional components contribute minimally to explaining further variance.

In Figure 10, the cumulative variance increases with each principal component. A horizontal dashed line at the 80% variance threshold indicates the target level of explained

variance. The vertical dotted line at the 10th principal component marks the number of components required to exceed this threshold. Specifically, the first ten principal components collectively explain approximately 80.59% of the total variance in the dataset.

6.3.3 Visualization of Principal Component Loadings

. The loadings were analyzed to identify the top five variables contributing to each of the first ten principal components.

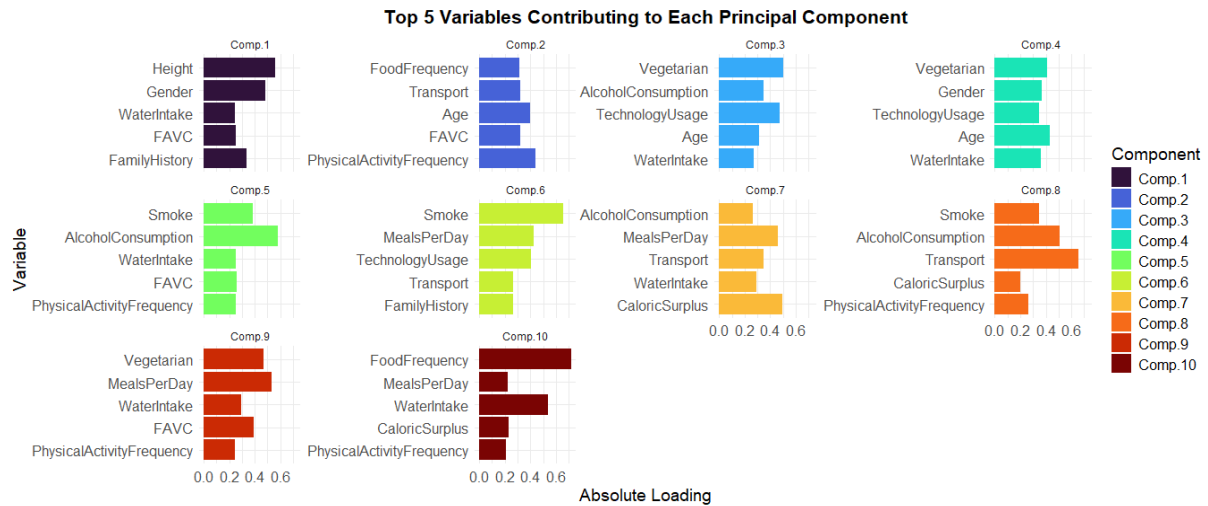


Figure 11: Top 5 Variables Contributing to Each of the First Ten Principal Components

The Principal Component Loadings Plot (Figure 11) highlights the variables with the highest absolute loadings for each component, indicating their significant contribution to the variance captured by the respective principal component. By focusing on the top five variables per component, the plot facilitates the identification of underlying patterns and relationships among the variables, providing deeper insights into the data structure.

6.3.4 Conclusion of PCA Analysis

The PCA effectively reduced the dataset's dimensionality from twelve original variables to ten principal components, explaining over 80% of the total variance. This simplification streamlines the data structure, facilitating subsequent clustering analyses while preserving the essential information needed for accurate and meaningful clustering.

6.4 PCA with k-Means Clustering

K-Means clustering is an unsupervised learning algorithm used to partition a dataset into distinct groups based on feature similarity. The objective is to minimize the within-cluster sum of squares (WSS), ensuring that data points within the same cluster are as similar as possible.

6.4.1 Elbow Method for Determining Optimal Clusters

To determine the optimal number of clusters (k), the Elbow Method was employed.

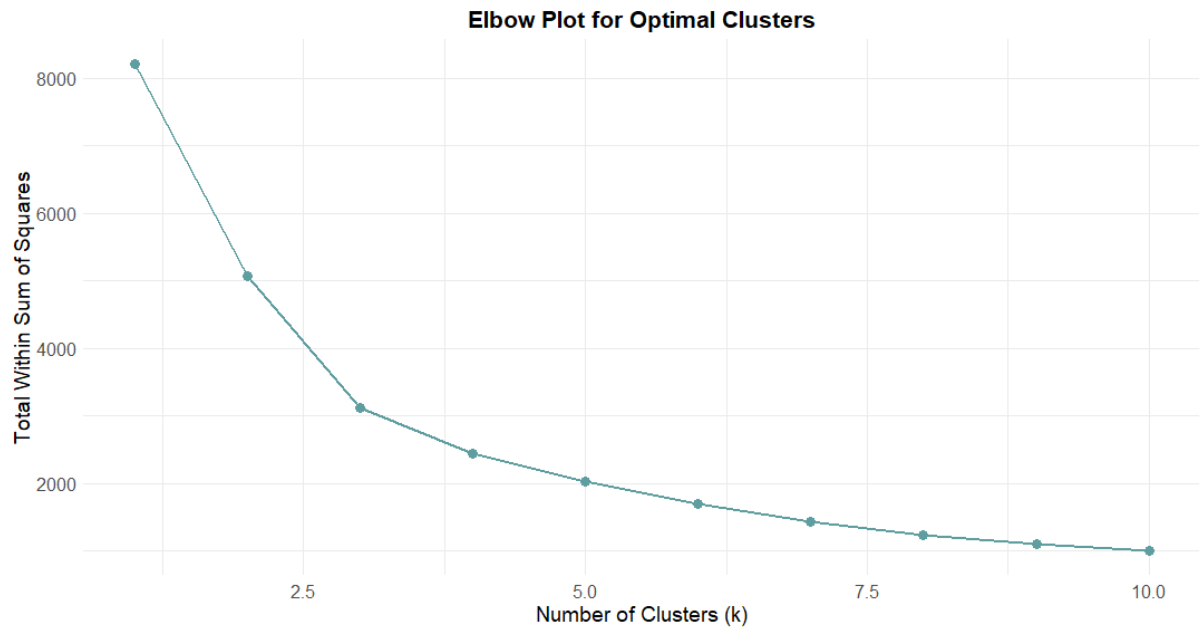


Figure 12: Elbow Plot for Optimal Number of Clusters in K-Means Clustering

The "elbow" point, indicating diminishing returns for additional clusters, was identified at $k = 3$.

6.4.2 K-Means Clustering Results

The K-Means algorithm was executed, and the results are visualized below.

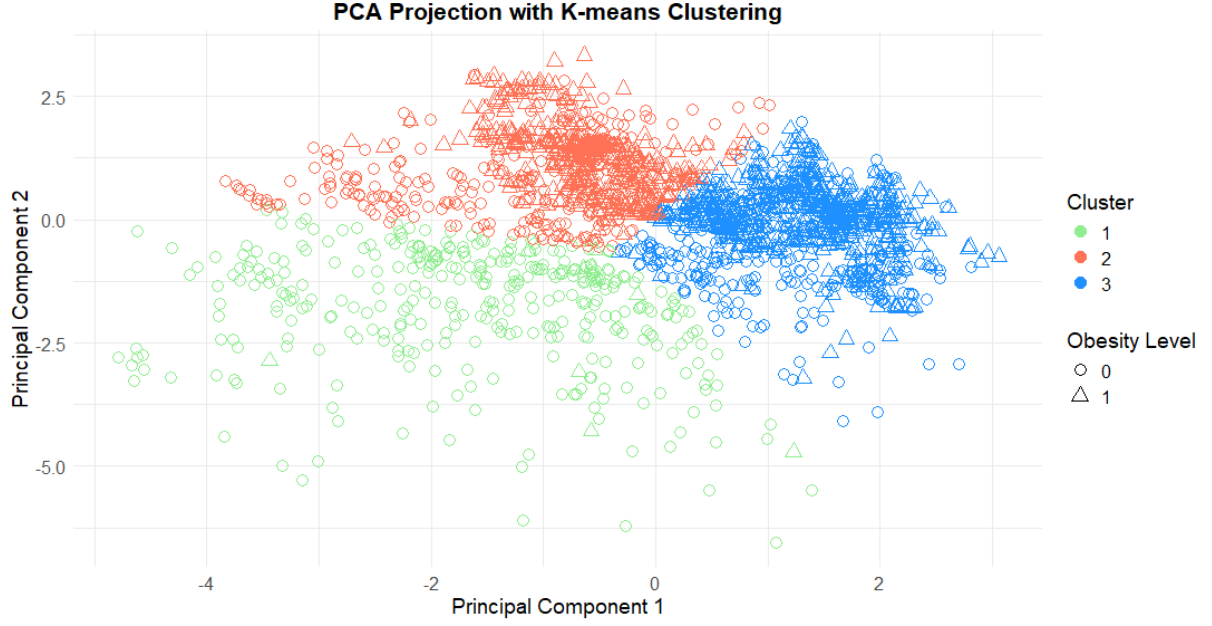


Figure 13: PCA Projection with K-Means Clustering

Table 9: Average Values of Each Variable for Each Cluster (K-Means)

Cluster	Gender	Age	Height	FamHist	FAVC	Veg	Meals	FooFreq	Smoke	Water	CalorSurp	PActivityFreq	TechUse
1	0.296	0.143	0.384	0.387	0.541	0.725	0.610	0.297	0.0359	0.431	0.254	0.489	0.411
2	0.146	0.265	0.352	0.876	0.941	0.771	0.481	0.0461	0.0119	0.418	0.00132	0.171	0.254
3	0.863	0.214	0.610	0.949	0.967	0.658	0.619	0.0295	0.0228	0.596	0.00311	0.412	0.363

We can observe that obese individuals are more prevalent in the **blue** and **red** clusters, while non-obese individuals are more commonly found in the **green** cluster. This pattern suggests that there are different lifestyle and dietary factors distinguish these clusters.

The **blue** cluster (**Cluster 3**) shows high values for **FamilyHistory**, frequent consumption of **FAVC** (high-calorie foods), and a relatively low **FoodFrequency**, indicating irregular meal patterns associated with obesity. Similarly, the **red** cluster (**Cluster 1**) exhibits moderate **FAVC** values and lower **PhysicalActivityFrequency**, reinforcing its association with obesity.

In contrast, the **green** cluster (**Cluster 2**) reflects healthier lifestyle choices, such as fewer **MealsPerDay**, lower **FAVC**. These traits align with a lower prevalence of obesity.

This distribution highlights the critical role of dietary habits, family history, and lifestyle behaviors in obesity risk.

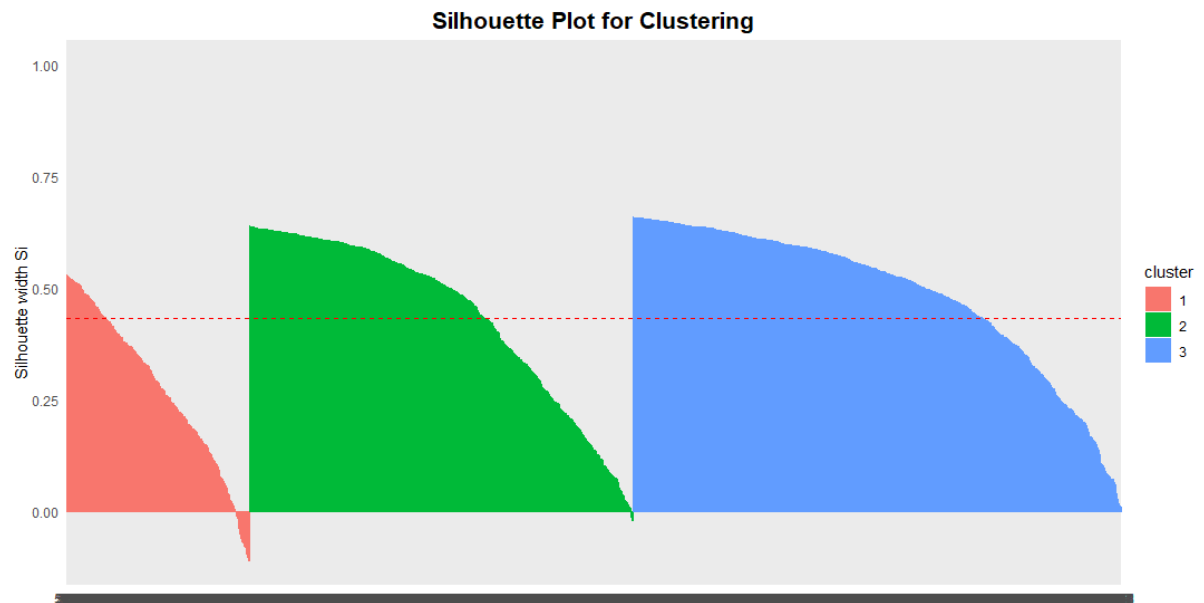


Figure 14: Silhouette Plot for K-Means Clustering

Average Silhouette Width: 0.433

6.5 PCA with hierarchical Clustering

Hierarchical clustering builds a tree-like structure (dendrogram) to represent the nested grouping of data points. Unlike K-Means, it does not require specifying the number of clusters upfront and can capture more intricate relationships within the data.

The Complete Linkage method was utilized to measure the dissimilarity between clusters, ensuring that clusters are formed based on the maximum distance between their members.

6.5.1 Dendrogram and Cluster Selection

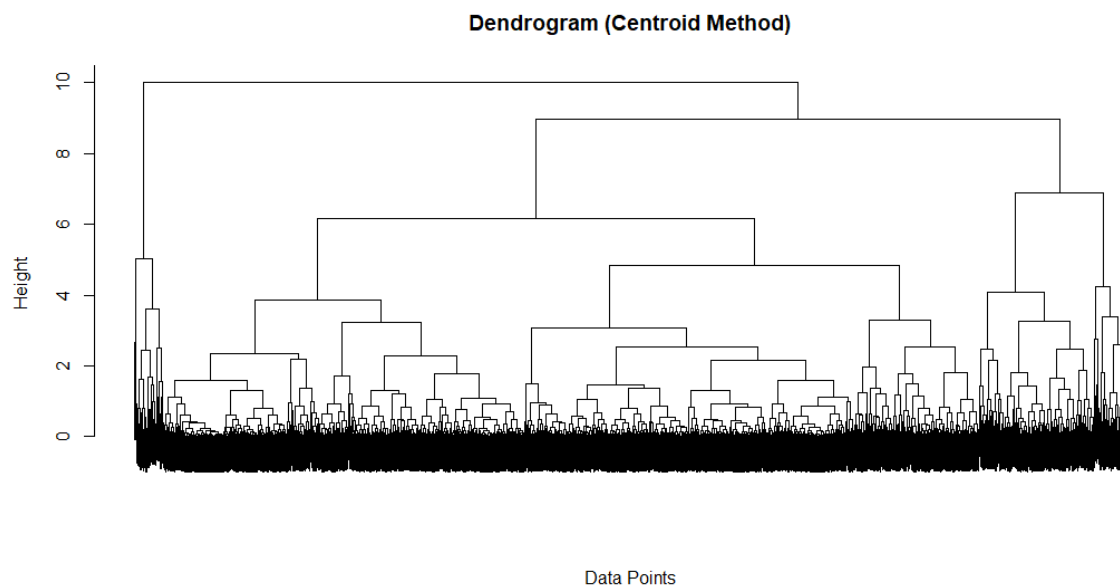


Figure 15: Dendrogram for Hierarchical Clustering

Figure 16 displays the dendrogram resulting from the hierarchical clustering analysis. A horizontal red dashed line was drawn at a height of approximately 6 to determine the cut-off for selecting clusters. Based on this threshold, $k = 4$ clusters were identified as optimal, capturing meaningful groupings within the data.

6.5.2 Hierarchical Clustering Results

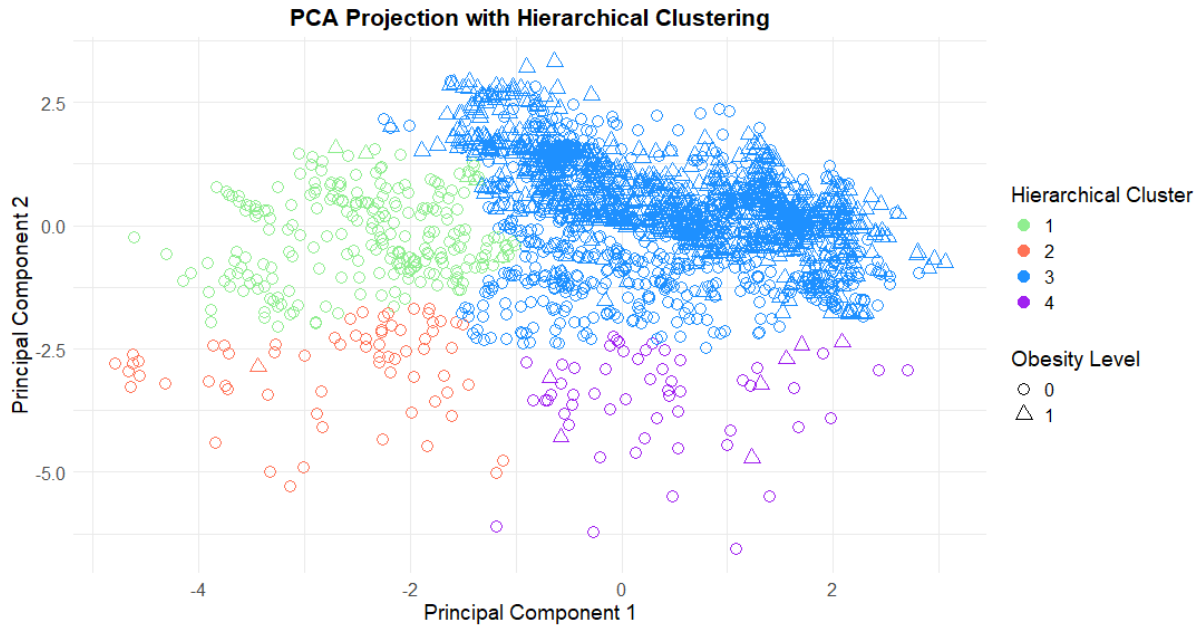


Figure 16: Dendrogram for Hierarchical Clustering

Table 10: Average Values of Each Variable for Each Cluster (K-Means)

Cluster	Gender	Age	Height	FamHist	FAVC	Veg	Meals	FooFreq	Smoke	Water	CalorSurp	PActivityFreq	TechUse
1	0.296	0.143	0.384	0.387	0.541	0.725	0.610	0.297	0.0359	0.431	0.254	0.489	0.411
2	0.146	0.265	0.352	0.876	0.941	0.771	0.481	0.0461	0.0119	0.418	0.00132	0.171	0.254
3	0.863	0.214	0.610	0.949	0.967	0.658	0.619	0.0295	0.0228	0.596	0.00311	0.412	0.363

We can observe that obese individuals are more prevalent in the **blue** and **purple** clusters, while non-obese individuals are more commonly found in the **red** and **green** clusters.

The **blue** cluster shows very high values for **FamilyHistory**, frequent consumption of **FAVC** (high-calorie foods), and low **FoodFrequency**, indicating patterns associated with a high obesity risk. The **purple** cluster (**Cluster 4**) has moderately high values for **Height**, **MealsPerDay**, and **FoodFrequency**, suggesting possible risk factors linked to dietary habits.

The **green** cluster (**Cluster 2**) displays the healthiest profile, with lower **FAVC**, higher **Vegetarian** prevalence, and balanced **MealsPerDay**, indicating a lower likelihood of obesity.

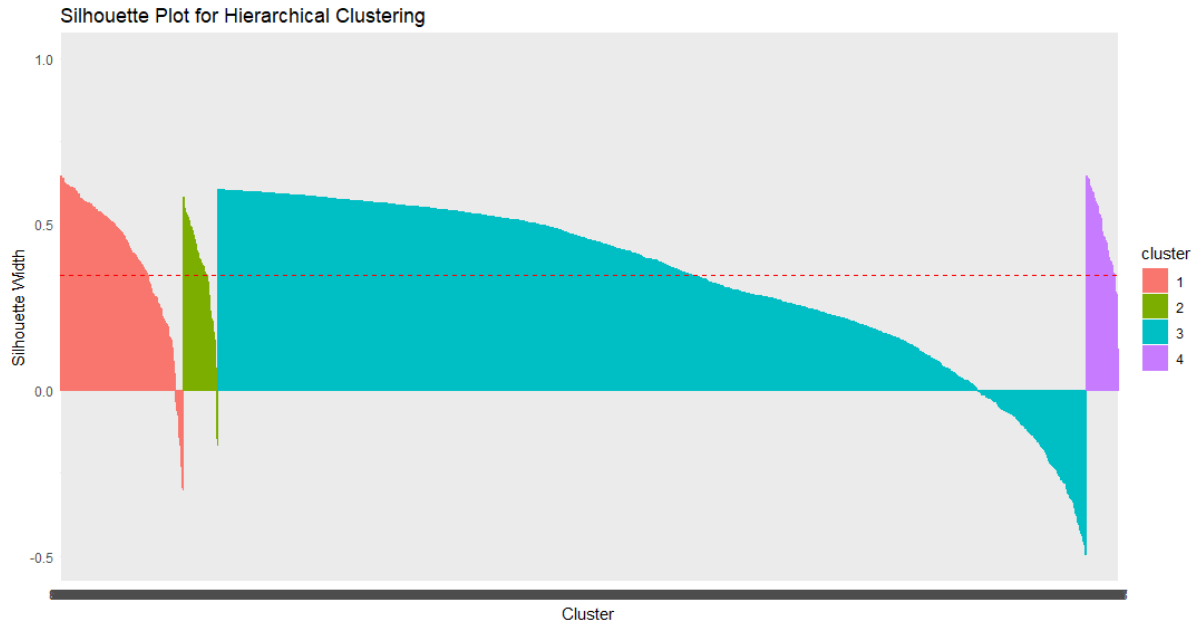


Figure 17: Silhouette Plot for Hierarchical Clustering

Average Silhouette Width: 0.345

6.6 Results

- Family History (FamilyHistory):**
 Both clustering methods highlight a strong link between a positive family history and obesity. Higher values in clusters associated with obesity suggest that genetic predisposition plays a critical role in determining obesity risk.
- Frequent High-Calorie Food Consumption (FAVC):**
 This variable consistently appears with high values in clusters linked to obesity, indicating that frequent consumption of calorie-dense foods is a major contributor to weight gain and obesity development.
- Food Frequency (FoodFrequency):**
 Interestingly, lower food frequency is associated with obesity in both clustering models. This pattern may reflect irregular eating habits or binge-eating behavior, which are commonly linked to weight gain.
- Physical Activity Frequency (PAFreq):**
 Lower physical activity is implied in clusters with higher obesity prevalence. This suggests that sedentary behavior is a significant risk factor contributing to obesity.

7 R Code

```
#-----  
# Load required libraries  
#-----  
library(ggplot2)  
library(dplyr)  
library(tidyr)  
library(reshape2)  
library(viridis)  
library(gridExtra)  
library(ggpubr)  
library(caret)  
library(randomForest)  
library(rpart)  
library(rpart.plot)  
library(xgboost)  
library(e1071)  
library(nnet)  
library(doParallel)  
library(RColorBrewer)  
library(psych)  
library(cluster)  
library(factoextra)  
library(knitr)  
library(kableExtra)  
library(openxlsx)  
library(scales)  
  
#-----  
# Parallelization Setup  
# Create a parallel cluster using all available cores except one for parallelized tasks.  
#-----  
cl <- makeCluster(detectCores() - 1)  
registerDoParallel(cl)  
  
#-----  
# Data Loading  
# Specify the path to the dataset and load it as a CSV file  
#-----  
file_path <- "C:\\Users\\Sara\\Desktop\\ObesityDataSet_raw_and_data_sinthetic.csv"  
dataset <- read.csv(file_path, sep = ",", header = TRUE, stringsAsFactors = FALSE)  
  
# Display the first few rows and structure to understand the data format  
head(dataset)  
str(dataset)  
  
#-----  
# Missing Values Handling  
# Count total missing values and remove rows if any  
#-----  
missing_values <- sum(is.na(dataset))  
cat("Total missing values:", missing_values, "\n")  
  
if (missing_values > 0) {  
  dataset <- na.omit(dataset)  
  cat("Rows with missing values removed.\n")  
} else {  
  cat("No missing values found.\n")  
}  
  
#-----  
# Duplicate Rows Handling  
# Check and remove duplicate rows if present  
#-----  
duplicate_rows <- sum(duplicated(dataset))  
cat("Total duplicate rows:", duplicate_rows, "\n")  
  
if (duplicate_rows > 0) {  
  dataset <- dataset[!duplicated(dataset), ]  
  cat("Duplicate rows removed.\n")  
} else {  
  cat("No duplicate rows found.\n")  
}
```

```

}

#-----
# Variable Renaming
# Rename columns for clarity and consistency
#-----
colnames(dataset) <- c(
  "Gender", "Age", "Height", "Weight",
  "FamilyHistory", "FAVC", "Vegetarian", "MealsPerDay",
  "FoodFrequency", "Smoke", "WaterIntake", "CaloricSurplus",
  "PhysicalActivityFrequency", "TechnologyUsage", "AlcoholConsumption", "Transport",
  "ObesityLevel"
)

#-----
# Data Transformation
# Convert categorical variables into numeric/binary encodings for modeling
#-----

# Gender: Convert "male" to 1 and otherwise 0
dataset$Gender <- ifelse(tolower(dataset$Gender) == "male", 1, 0)

# Binary variables: Convert "yes"/"frequently" to 1 and otherwise 0
binary_vars <- c("FamilyHistory", "FAVC", "Smoke", "CaloricSurplus")
dataset[binary_vars] <- lapply(dataset[binary_vars], function(x) {
  ifelse(tolower(x) %in% c("yes", "frequently"), 1, 0)
})

# AlcoholConsumption: Encode "no"=0, "sometimes"=1, "frequently"=2, missing replaced by median
dataset$AlcoholConsumption <- case_when(
  tolower(dataset$AlcoholConsumption) == "no" ~ 0,
  tolower(dataset$AlcoholConsumption) == "sometimes" ~ 1,
  tolower(dataset$AlcoholConsumption) == "frequently" ~ 2,
  TRUE ~ NA_real_
)

if (any(is.na(dataset$AlcoholConsumption))) {
  median_alcohol <- median(dataset$AlcoholConsumption, na.rm = TRUE)
  dataset$AlcoholConsumption[is.na(dataset$AlcoholConsumption)] <- median_alcohol
  cat("Missing values in AlcoholConsumption replaced with median:", median_alcohol, "\n")
}

# FoodFrequency: Encode categorical frequency into numeric scale (0 to 4)
dataset$FoodFrequency <- case_when(
  tolower(dataset$FoodFrequency) == "never" ~ 0,
  tolower(dataset$FoodFrequency) == "rarely" ~ 1,
  tolower(dataset$FoodFrequency) == "sometimes" ~ 2,
  tolower(dataset$FoodFrequency) == "frequently" ~ 3,
  tolower(dataset$FoodFrequency) == "always" ~ 4,
  TRUE ~ NA_real_
)

if (any(is.na(dataset$FoodFrequency))) {
  median_foodfreq <- median(dataset$FoodFrequency, na.rm = TRUE)
  dataset$FoodFrequency[is.na(dataset$FoodFrequency)] <- median_foodfreq
  cat("Missing values in FoodFrequency replaced with median:", median_foodfreq, "\n")
}

# Transport: Encode transportation modes into binary categories (active vs. not active)
dataset$Transport <- ifelse(tolower(dataset$Transport) %in% c("public_transportation", "automobile"), 0,
  ifelse(tolower(dataset$Transport) %in% c("bike", "walking"), 1, NA))

if (any(is.na(dataset$Transport))) {
  dataset$Transport[is.na(dataset$Transport)] <- 0
  cat("Missing values in Transport replaced with 0.\n")
}

# ObesityLevel: Group obesity levels into binary outcome (0=NonObese, 1=Obese)
dataset$ObesityLevel <- case_when(
  tolower(dataset$ObesityLevel) %in% c("insufficient_weight", "underweight", "normal_weight",
    "overweight_level_i", "overweight_level_ii") ~ 0,
  tolower(dataset$ObesityLevel) %in% c("obesity_type_i", "obesity_type_ii", "obesity_type_iii") ~ 1,
  TRUE ~ NA_real_
)

```

```

)

if (any(is.na(dataset$ObesityLevel))) {
  median_obesity <- median(dataset$ObesityLevel, na.rm = TRUE)
  dataset$ObesityLevel[is.na(dataset$ObesityLevel)] <- median_obesity
  cat("Missing values in ObesityLevel replaced with median:", median_obesity, "\n")
}

#-----
# Initial Correlation and Model Check
# Check correlation of Weight with ObesityLevel, fit a simple logistic model and measure accuracy
#-----
correlation <- cor(dataset$Weight, dataset$ObesityLevel, method = "pearson")
cat("Correlation between Weight and ObesityLevel:", round(correlation, 4), "\n")

model_lr_initial <- glm(ObesityLevel ~ Weight, data = dataset, family = binomial)
summary(model_lr_initial)

predicted_probs_lr_initial <- predict(model_lr_initial, type = "response")
predicted_classes_lr_initial <- ifelse(predicted_probs_lr_initial > 0.5, 1, 0)
accuracy_lr_initial <- mean(predicted_classes_lr_initial == dataset$ObesityLevel)
cat("Initial Logistic Regression Accuracy with Weight as predictor:", round(accuracy_lr_initial, 4), "\n")

# Remove Weight due to high correlation and potential data leakage issues
dataset <- dataset %>% select(-Weight)
cat("Weight variable removed due to high correlation with ObesityLevel.\n")

#-----
# Data Scaling
# Apply Min-Max scaling to numeric predictors (excluding the target variable)
#-----
numeric_vars <- sapply(dataset, is.numeric)
numeric_vars["ObesityLevel"] <- FALSE

min_max_scale <- function(x) {
  return((x - min(x)) / (max(x) - min(x)))
}

dataset_scaled <- dataset
dataset_scaled[, numeric_vars] <- lapply(dataset_scaled[, numeric_vars], min_max_scale)

# Convert the target variable to a factor with descriptive labels
dataset_scaled$ObesityLevel <- factor(dataset_scaled$ObesityLevel, levels = c(0, 1), labels = c("NonObese", "Obese"))

# Display scaled dataset details
cat("\n**First Few Rows of the Scaled Dataset:**\n")
print(head(dataset_scaled))
cat("\n**Structure of the Scaled Dataset:**\n")
print(str(dataset_scaled))

#-----
# Descriptive Statistics
# Compute summary statistics for numeric variables
#-----
# Target variable distribution
target_freq <- dataset_scaled %>%
  count(ObesityLevel) %>%
  mutate(Percentage = n / sum(n) * 100)

# Pie Chart for target distribution
pie_chart <- ggplot(target_freq, aes(x = "", y = Percentage, fill = ObesityLevel)) +
  geom_bar(stat = "identity", width = 1) +
  coord_polar(theta = "y") +
  theme_void() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold")) +
  labs(title = "Distribution of Obesity Levels", fill = "Obesity Level") +
  scale_fill_manual(values = c("darkorange", "cadetblue2")) +
  geom_text(aes(label = paste0(round(Percentage, 1), "%"),
    position = position_stack(vjust = 0.5),
    color = "white", size = 4))

print(pie_chart)

```

```

#-----
# Correlation Analysis
# Compute and visualize correlation matrix for numeric predictors
#-----
# Including ObesityLevel in correlation matrix
numeric_data_adjusted <- dataset_scaled
numeric_data_adjusted$ObesityLevel <- as.numeric(dataset_scaled$ObesityLevel) - 1

# Compute the correlation matrix including ObesityLevel
cor_matrix <- cor(numeric_data_adjusted, use = "pairwise.complete.obs")

# Reshape the correlation matrix into a long format
cor_matrix_long <- melt(cor_matrix, varnames = c("Variable1", "Variable2"),
                        value.name = "Correlation")

# Create the heatmap plot
heatmap_plot <- ggplot(data = cor_matrix_long,
                      aes(x = Variable1, y = Variable2, fill = Correlation)) +
  geom_tile(color = "white") +
  scale_fill_viridis_c(direction = 1, option = "turbo") +
  geom_text(aes(label = round(Correlation, 2)), color = "white", size = 3) +
  labs(title = " Correlation Heatmap",
       x = "Variables",
       y = "Variables",
       fill = "Correlation") +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1, size = 10),
    axis.text.y = element_text(size = 10),
    plot.title = element_text(face = "bold", hjust = 0.5, size = 14)
  )

# Print the heatmap plot
print(heatmap_plot)

#-----
# Supervised
# Data Splitting
# Split data into training (80%) and testing (20%) sets for model evaluation
#-----
set.seed(111)
indexes <- createDataPartition(dataset_scaled$ObesityLevel, p = 0.80, list = FALSE)
train_set <- dataset_scaled[indexes, ]
test_set <- dataset_scaled[-indexes, ]

#-----
# Modeling - Logistic Regression
# Fit a logistic regression model on all predictors
#-----
model_lr <- glm(ObesityLevel ~ ., data = train_set, family = binomial)
summary(model_lr)

predicted_probs_lr <- predict(model_lr, test_set, type = "response")
predicted_classes_lr <- ifelse(predicted_probs_lr > 0.5, "Obese", "NonObese")
predicted_classes_lr <- factor(predicted_classes_lr, levels = c("NonObese", "Obese"))
actual_classes_lr <- factor(test_set$ObesityLevel, levels = c("NonObese", "Obese"))

confusionMatrix_lr <- confusionMatrix(predicted_classes_lr, actual_classes_lr)

cat("\n**Logistic Regression Confusion Matrix:**\n")
print(confusionMatrix_lr$table)

cat("\n**Logistic Regression Performance Metrics:**\n")
cat("Accuracy:", round(confusionMatrix_lr$overall['Accuracy'], 4), "\n")
cat("Precision:", round(confusionMatrix_lr$byClass['Precision'], 4), "\n")
cat("Recall:", round(confusionMatrix_lr$byClass['Recall'], 4), "\n")
cat("F1 Score:", round(confusionMatrix_lr$byClass['F1'], 4), "\n")

#-----
# Modeling - K-Nearest Neighbors (KNN)
# Use 10-fold cross-validation to tune k, with scaling and centering
#-----

```

```

train_set_knn <- train_set
test_set_knn <- test_set

train_set_knn$ObesityLevel <- factor(train_set_knn$ObesityLevel, levels = c("NonObese", "Obese"))
test_set_knn$ObesityLevel <- factor(test_set_knn$ObesityLevel, levels = c("NonObese", "Obese"))

train_control_knn <- trainControl(
  method = "cv",
  number = 10,
  savePredictions = "final",
  classProbs = TRUE,
  summaryFunction = defaultSummary
)

metric <- "Accuracy"

set.seed(111)
knn_model <- train(
  ObesityLevel ~ .,
  data = train_set_knn,
  method = "knn",
  trControl = train_control_knn,
  tuneLength = 20,
  metric = metric,
  preProcess = c("center", "scale")
)

cat("\n**KNN Model Details:**\n")
print(knn_model)

knn_predictions <- predict(knn_model, newdata = test_set_knn)
knn_results <- confusionMatrix(knn_predictions, test_set_knn$ObesityLevel)

cat("\n**KNN Confusion Matrix:**\n")
print(knn_results$table)

cat("\n**KNN Performance Metrics:**\n")
cat("Accuracy:", round(knn_results$overall['Accuracy'], 4), "\n")
cat("Precision:", round(knn_results$byClass['Precision'], 4), "\n")
cat("Recall:", round(knn_results$byClass['Recall'], 4), "\n")
cat("F1 Score:", round(knn_results$byClass['F1'], 4), "\n")

#-----
# Modeling - Decision Tree
# Optimize max depth using cross-validation
#-----
train_control_tree <- trainControl(method = "cv", number = 10, allowParallel = TRUE)
tune_grid_tree <- expand.grid(maxdepth = c(3, 5, 7, 10, 15))

tree_model <- train(ObesityLevel ~ ., data = train_set, method = "rpart2",
  trControl = train_control_tree, tuneGrid = tune_grid_tree)

cat("\n**Decision Tree Model Details:**\n")
print(tree_model)

best_maxdepth <- tree_model$bestTune$maxdepth
cat("Optimal maxdepth:", best_maxdepth, "\n")

final_tree <- tree_model$finalModel

# Visualize the decision tree
rpart.plot(final_tree, type = 3, extra = 101, under = TRUE, facLen = 0,
  main = "Decision Tree with Optimized Maxdepth")

# Evaluate on test set
tree_predictions <- predict(tree_model, newdata = test_set)
cm_tree <- confusionMatrix(tree_predictions, test_set$ObesityLevel)
cat("\n**Decision Tree Confusion Matrix:**\n")
print(cm_tree$table)

accuracy_tree <- cm_tree$overall["Accuracy"]
precision_tree <- cm_tree$byClass["Precision"]
recall_tree <- cm_tree$byClass["Recall"]

```



```

F1_tree <- 2 * (precision_tree * recall_tree) / (precision_tree + recall_tree)

cat("\n**Decision Tree Performance Metrics:**\n")
cat("Accuracy:", round(accuracy_tree, 4), "\n")
cat("Precision:", round(precision_tree, 4), "\n")
cat("Recall:", round(recall_tree, 4), "\n")
cat("F1 Score:", round(F1_tree, 4), "\n")

#-----
# Modeling - Random Forest
# Tune 'mtry' parameter using 5-fold cross-validation
#-----
train_control_rf <- trainControl(method = "cv", number = 5, allowParallel = TRUE)
tune_grid_rf <- expand.grid(mtry = c(2, 4, 6, 8))

set.seed(123)
rf_model <- train(ObesityLevel ~ ., data = train_set, method = "rf",
                 trControl = train_control_rf, tuneGrid = tune_grid_rf, ntree = 500)

rf_predictions <- predict(rf_model, newdata = test_set)
cm_rf <- confusionMatrix(rf_predictions, test_set$ObesityLevel)

accuracy_rf <- cm_rf$overall["Accuracy"]
precision_rf <- cm_rf$byClass["Precision"]
recall_rf <- cm_rf$byClass["Recall"]
F1_rf <- 2 * (precision_rf * recall_rf) / (precision_rf + recall_rf)

cat("\n**Random Forest Performance Metrics:**\n")
cat("Accuracy:", round(accuracy_rf, 4), "\n")
cat("Precision:", round(precision_rf, 4), "\n")
cat("Recall:", round(recall_rf, 4), "\n")
cat("F1 Score:", round(F1_rf, 4), "\n")

# Variable importance from Random Forest
var_imp_rf <- varImp(rf_model, scale = FALSE)
print(var_imp_rf)

var_imp_df <- data.frame(
  Variable = rownames(var_imp_rf$importance),
  Importance = var_imp_rf$importance[,1]
)

var_imp_df <- var_imp_df %>%
  arrange(desc(Importance)) %>%
  top_n(20, Importance) %>%
  mutate(Variable = factor(Variable, levels = rev(unique(Variable))))

varimp_plot <- ggplot(var_imp_df, aes(x = Variable, y = Importance)) +
  geom_bar(stat = "identity", fill = "cadetblue") +
  coord_flip() +
  labs(title = "Variable Importance - Random Forest",
       x = "Variables",
       y = "Importance") +
  theme_minimal()

print(varimp_plot)

#-----
# Modeling - XGBoost
# Comprehensive grid search on multiple hyperparameters
#-----
train_control_xgb <- trainControl(
  method = "cv",
  number = 5,
  verboseIter = TRUE,
  allowParallel = TRUE
)

tune_grid_xgb <- expand.grid(
  nrounds = c(50, 100, 150),
  max_depth = c(3, 6, 9),
  eta = c(0.01, 0.1),
  gamma = c(0, 0.1),

```

```

    colsample_bytree = c(0.7, 0.9),
    min_child_weight = c(1, 5),
    subsample = c(0.7, 0.9)
)

set.seed(111)
xgb_tuned_model <- train(
  ObesityLevel ~ .,
  data = train_set,
  method = "xgbTree",
  trControl = train_control_xgb,
  tuneGrid = tune_grid_xgb,
  verbose = FALSE
)

cat("\n**XGBoost Best Tuning Parameters:**\n")
print(xgb_tuned_model$bestTune)

xgb_predictions <- predict(xgb_tuned_model, newdata = test_set)
cm_xgb <- confusionMatrix(xgb_predictions, test_set$ObesityLevel)

cat("\n**XGBoost Confusion Matrix:**\n")
print(cm_xgb$table)

accuracy_xgb <- cm_xgb$overall["Accuracy"]
precision_xgb <- cm_xgb$byClass["Precision"]
recall_xgb <- cm_xgb$byClass["Recall"]
F1_xgb <- 2 * (precision_xgb * recall_xgb) / (precision_xgb + recall_xgb)

cat("\n**XGBoost Performance Metrics:**\n")
cat("Accuracy:", round(accuracy_xgb, 4), "\n")
cat("Precision:", round(precision_xgb, 4), "\n")
cat("Recall:", round(recall_xgb, 4), "\n")
cat("F1 Score:", round(F1_xgb, 4), "\n")

#-----
# Model Comparison
# Compare the performance of all tested models using accuracy, precision, recall, and F1
#-----
metrics_df <- data.frame(
  Model = c("Logistic Regression", "KNN", "Decision Tree", "Random Forest", "XGBoost"),
  Accuracy = c(round(confusionMatrix_lr$overall['Accuracy'], 4),
               round(knn_results$overall['Accuracy'], 4),
               round(accuracy_tree, 4),
               round(accuracy_rf, 4),
               round(accuracy_xgb, 4)),
  Precision = c(round(confusionMatrix_lr$byClass['Precision'], 4),
               round(knn_results$byClass['Precision'], 4),
               round(precision_tree, 4),
               round(precision_rf, 4),
               round(precision_xgb, 4)),
  Recall = c(round(confusionMatrix_lr$byClass['Recall'], 4),
             round(knn_results$byClass['Recall'], 4),
             round(recall_tree, 4),
             round(recall_rf, 4),
             round(recall_xgb, 4)),
  F1_Score = c(round(confusionMatrix_lr$byClass['F1'], 4),
               round(knn_results$byClass['F1'], 4),
               round(F1_tree, 4),
               round(F1_rf, 4),
               round(F1_xgb, 4))
)

cat("\n**Model Comparison Table:**\n")
kable(metrics_df, digits = 4, caption = "Model Comparison") %>%
  kable_styling(full_width = FALSE, position = "center") %>%
  row_spec(0, background = "#d4edda", bold = TRUE)

#-----
# Unsupervised
# Perform PCA to reduce dimensionality and explore cluster structures
#-----

```

```

pca_data <- dataset_scaled %>% select(-ObesityLevel)
pca_train <- princomp(pca_data, cor = TRUE)

cat("\n**PCA Summary:**\n")
print(summary(pca_train))

#-----
# Scree Plot of Cumulative Explained Variance
#-----

# Extract cumulative explained variance manually to avoid locale issues
cumulative_variance <- cumsum(pca_train$sdev^2) / sum(pca_train$sdev^2)

# Create a data frame for plotting
scree_data <- data.frame(
  Component = 1:length(cumulative_variance),
  CumulativeVariance = cumulative_variance
)

# Generate Scree Plot
scree_plot <- ggplot(scree_data, aes(x = Component, y = CumulativeVariance)) +
  geom_point(color = "steelblue", size = 3) +
  geom_line(color = "steelblue") +
  geom_vline(xintercept = 10, linetype = "dashed", color = "darkorange", size = 1) +
  geom_hline(yintercept = 0.80, linetype = "dashed", color = "black", size = 1) +
  annotate("text", x = 10, y = min(cumulative_variance),
    label = "PC10", color = "black", vjust = -27, size = 5) +
  annotate("text", x = max(scree_data$Component),
    y = 0.80, label = "80%", color = "black", vjust = -0.5, hjust = 1.1, size = 5) +
  scale_x_continuous(breaks = scree_data$Component) +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(
    title = "Scree Plot: Cumulative Explained Variance",
    x = "Principal Component",
    y = "Cumulative Explained Variance"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
    axis.title = element_text(size = 14),
    axis.text = element_text(size = 12)
  )

# Display the Scree Plot
print(scree_plot)

# Extract loadings and identify top contributing variables to each principal component
loadings <- pca_train$loadings
loadings_matrix <- as.matrix(unclass(loadings))
selected_components <- paste0("Comp.", 1:10)
selected_components <- selected_components[selected_components %in% colnames(loadings_matrix)]
loadings_selected <- loadings_matrix[, selected_components, drop = FALSE]
loadings_df <- as.data.frame(loadings_selected)
loadings_df$Variable <- rownames(loadings_selected)

top_variables <- loadings_df %>%
  pivot_longer(
    cols = starts_with("Comp."),
    names_to = "Component",
    values_to = "Loading"
  ) %>%
  group_by(Component) %>%
  arrange(desc(abs>Loading))) %>%
  slice_head(n = 5) %>%
  ungroup() %>%
  mutate(
    Component_Num = as.numeric(gsub("Comp\\.\\.", "", Component)),
    Component = factor(Component, levels = paste0("Comp.", 1:10))
  )

ggplot(top_variables, aes(x = reorder(Variable, abs>Loading)), y = abs>Loading), fill = Component)) +
  geom_bar(stat = "identity") +
  coord_flip() +

```

```

labs(
  title = "Top 5 Variables Contributing to Each Principal Component",
  x = "Variable",
  y = "Absolute Loading"
) +
theme_minimal() +
theme(
  plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
  axis.title.x = element_text(size = 14),
  axis.title.y = element_text(size = 14),
  axis.text = element_text(size = 12),
  legend.title = element_text(size = 14),
  legend.text = element_text(size = 12)
) +
facet_wrap(~ Component, scales = "free_y") +
scale_fill_viridis_d(option = "turbo")

#-----
# K-Means Clustering
# Perform k-means clustering on PCA-transformed data and evaluate with silhouette scores
#-----

obesity_level <- dataset_clustering$ObesityLevel
data_pca <- dataset_clustering[, -ncol(dataset_clustering)]

pca_result <- prcomp(data_pca, scale. = TRUE)

explained_variance <- summary(pca_result)$importance[3, ]
print("Cumulative Proportion of Variance Explained:")
print(explained_variance)

n_components <- 10
data_reduced <- as.data.frame(pca_result$x[, 1:n_components])
# For plotting simplicity, rename columns to PC1 and PC2 only
colnames(data_reduced)[1:2] <- c("PC1", "PC2")

data_reduced$ObesityLevel <- factor(obesity_level)
data_reduced$ID <- 1:nrow(data_reduced)

# Determine optimal k using Elbow method
set.seed(123)
wss <- numeric(10)
for (k in 1:10) {
  kmeans_result <- kmeans(data_reduced[, c("PC1", "PC2")], centers = k, nstart = 25)
  wss[k] <- kmeans_result$tot.withinss
}

elbow_plot <- ggplot(data.frame(k = 1:10, wss = wss), aes(x = k, y = wss)) +
  geom_point(size = 3, color = "cadetblue") +
  geom_line(size = 1, color = "cadetblue") +
  labs(title = "Elbow Plot for Optimal Clusters",
       x = "Number of Clusters (k)",
       y = "Total Within Sum of Squares") +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
    axis.title = element_text(size = 14),
    axis.text = element_text(size = 12)
  )

print(elbow_plot)

# Assume optimal_k = 3 based on elbow plot
optimal_k <- 3
set.seed(123)
kmeans_result <- kmeans(data_reduced[, c("PC1", "PC2")], centers = optimal_k, nstart = 25)

data_reduced$Cluster <- factor(kmeans_result$cluster)

# Plot clusters in PCA space
cluster_plot <- ggplot(data_reduced, aes(x = PC1, y = PC2, color = Cluster, shape = ObesityLevel)) +
  geom_point(alpha = 1, size = 3.5) +
  scale_color_manual(values = c("lightgreen", "coral1", "dodgerblue")) +

```

```

scale_shape_manual(values = c(1, 2, 3, 4, 5, 6, 15, 17, 18, 19)) +
theme_minimal() +
theme(
  plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
  axis.title = element_text(size = 14),
  axis.text = element_text(size = 12),
  legend.title = element_text(size = 14),
  legend.text = element_text(size = 12)
) +
labs(
  title = "PCA Projection with K-means Clustering",
  x = "Principal Component 1",
  y = "Principal Component 2",
  color = "Cluster",
  shape = "Obesity Level"
)

print(cluster_plot)

# Evaluate clustering quality using silhouette
silhouette_scores <- silhouette(kmeans_result$cluster, dist(data_reduced[, c("PC1", "PC2")]))
silhouette_plot <- fviz_silhouette(silhouette_scores) +
  labs(title = "Silhouette Plot for Clustering") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, size = 16, face = "bold"))
print(silhouette_plot)

avg_silhouette_width <- mean(silhouette_scores[, 3])
print(paste("Average Silhouette Width:", round(avg_silhouette_width, 4)))

# Compute average value of each variable for each cluster (K-Means)
data_with_clusters <- cbind(dataset_clustering[, -ncol(dataset_clustering)], Cluster = kmeans_result$cluster)
cluster_summary <- data_with_clusters %>%
  group_by(Cluster) %>%
  summarise(across(everything(), mean, na.rm = TRUE))
print("Average values of each variable for each cluster (K-Means):")
print(cluster_summary)

#-----
# Hierarchical Clustering
# Perform hierarchical clustering on reduced PCA data
#-----

obesity_level <- dataset_clustering$ObesityLevel
data_pca <- dataset_clustering[, -ncol(dataset_clustering)]

pca_result <- prcomp(data_pca, scale. = TRUE)

explained_variance <- summary(pca_result)$importance[3, ]
print("Cumulative Proportion of Variance Explained:")
print(explained_variance)

n_components <- 10
data_reduced <- as.data.frame(pca_result$x[, 1:n_components])
colnames(data_reduced) <- paste0("PC", 1:n_components)

data_reduced$ObesityLevel <- factor(obesity_level)
data_reduced$ID <- 1:nrow(data_reduced)

# Compute dissimilarity matrix and perform hierarchical clustering
dissimilarity_matrix <- dist(data_reduced[, c("PC1", "PC2")])
hierarchical_clustering <- hclust(dissimilarity_matrix, method = "complete")

plot(hierarchical_clustering, labels = FALSE, main = "Dendrogram (Complete Linkage)",
  xlab = "Data Points", ylab = "Height", sub = "")
abline(h = 20, col = "red", lty = 2)

optimal_clusters <- cutree(hierarchical_clustering, k = 4)
data_reduced$Cluster_HC <- factor(optimal_clusters)

# Plot hierarchical clusters
hc_cluster_plot <- ggplot(data_reduced, aes(x = PC1, y = PC2, color = Cluster_HC, shape = ObesityLevel)) +
  geom_point(alpha = 1, size = 3.5) +

```

```

scale_color_manual(values = c("lightgreen", "coral1", "dodgerblue", "purple")) +
scale_shape_manual(values = c(1, 2, 3, 4, 5, 6, 15, 17, 18, 19)) +
theme_minimal() +
theme(
  plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
  axis.title = element_text(size = 14),
  axis.text = element_text(size = 12),
  legend.title = element_text(size = 14),
  legend.text = element_text(size = 12)
) +
labs(
  title = "PCA Projection with Hierarchical Clustering",
  x = "Principal Component 1",
  y = "Principal Component 2",
  color = "Hierarchical Cluster",
  shape = "Obesity Level"
)

print(hc_cluster_plot)

# Compute average value of each variable for each hierarchical cluster
hc_cluster_summary <- cbind(data_pca, Cluster_HC = optimal_clusters) %>%
  group_by(Cluster_HC) %>%
  summarise(across(everything(), mean, na.rm = TRUE))
print("Average values of each variable for each hierarchical cluster:")
print(hc_cluster_summary)

# Evaluate hierarchical clustering with silhouette scores
silhouette_scores_hc <- silhouette(optimal_clusters, dist(data_reduced[, c("PC1", "PC2")]))
avg_silhouette_width_hc <- mean(silhouette_scores_hc[, 3])
print(paste("Average Silhouette Width for Hierarchical Clustering:", round(avg_silhouette_width_hc, 4)))

silhouette_plot_hc <- fviz_silhouette(silhouette_scores_hc) +
  labs(title = "Silhouette Plot for Hierarchical Clustering",
       x = "Cluster",
       y = "Silhouette Width") +
  theme_minimal()
print(silhouette_plot_hc)

#-----
# Cleanup and Stop Parallel Processing
#-----
stopCluster(cl)
registerDoSEQ()
cat("\nParallel processing cluster stopped.\n")

```